



Computer Programming (CS F111)
Comprehensive Exam (Regular – Closed Book Exam)
First Semester: 2024-25
Instructors: Dr. Asish Bera, Dr. Bharat Richhariya, Dr. Vinti Agarwal & Dr. Tanmaya Mahapatra (IC)
Department of Computer Science and Information Systems

NAME AND ID: _____

Important Information:

- Write your name and ID legibly on every sheet of the exam booklet in the space provided.
- Please check the completeness of the exam booklet (9 double-sided pages)
- Use only blue or black pen to answer the questions; Pencils, green and red pens are not allowed!
- You must answer the questions in the exam booklet. No additional sheets will be provided!
- Working time is 120 minutes for 3 questions of 75 points!
- On any attempt to cheat: the exam will be assessed with a 0 and you would be barred from makeup.
- Permitted writing aids: none — closed book exam!

Question:	1	2	3	Total
Points:	19	34	22	75
Score:				

Question 1: Marking Scheme

- If the answer does not match with the output printed in green in this document for all sub-questions of Q1 → Award ZERO (0)
- If the output is printed in two new lines and this is not matching → Award ZERO (0)

1. What will be the output of the given codes?

Question 1: 19 pts

(a) What will be the output of the given code?

[02 pts]

```
1      #include <stdio.h>
2      void main(){
3          FILE *fp;
4          fp = fopen("myfile.txt","w+");
5          fputs("Comprehensive Examination", fp);
6          fseek(fp,15, SEEK_SET);
7          printf("%ld\t", ftell(fp));
8          char c = fgetc(fp);
9          while (c != EOF)
10             { printf ("%c", c);
11               c = fgetc(fp); }
12          fclose(fp);
13      }
```

Solution
15 xamination

(b) What will be the output of the given code?

[02 pts]

```
1      #include<stdio.h>
2      #define fun(x) 20
3      #define foo(x) (x>0)? 10:(fun(x-2)-x)
4      int main()
5      {
6          printf ("foo=%d\n", foo(-5));
7          return 0;
8      }
```

Solution
foo=25

(c) What will be the output of the given code?

[02 pts]

```
1      #include<stdio.h>
2      int main()
3      {
4          int a[5]={11,22,33,44,55}, *p;
5          p=a;
6          printf(" %d", ++*p++);
7          printf(" %d", *p);
8          return 0;
9      }
```

.....

.....

Solution

12 22

(d) Write the output for the scenarios:

```
1      #include <stdio.h>
2      #pragma pack(1)
3      struct x{
4          int a, d;   char b;};
5      union z{   struct x s;   int a;};
6
7      int main ( ){
8          union z u;
9          printf("size=%ld\n", sizeof(u));
10         return 0;
11     }
```

i. what will be the output **with** #pragma pack(1)?

[01 pts]

.....

.....

Solution

size=9

ii. what will be the output **without** #pragma pack(1)?

[01 pts]

.....

.....

Solution

size=12

(e) What will be the output of the given code?

[03 pts]

```
1      #include<stdio.h>
2      int main(int argc, char *argv[ ])
3      {
4          int x = 5, *y, z[2] = {22, 33}, **p;
5          y=&x;
6          p =&y;
7          **p = 10;
8          y=z;
9          *(z + 1) += x;
10         printf("%d, %d, %d, %d\n", x, +++*p, z[0], z[1]);
11         return 0;
12     }
```

.....

.....

Solution

10, 23, 22, 43

(f) What will be the output of the given code?

[02 pts]

```
1      #include<stdio.h>
2      enum fun{A, E, I, O, U};
3
4      int main()
5      {
6          printf("%d",A+I+5);
7          return 0;
8      }
```

.....

.....

Solution

7

(g) What will be the output of the given code?

[02 pts]

```
1      #include <stdio.h>
2      int main()
3      {
4          int A = 10;
5          A = A + 'B';
6          printf("%d \t%d",A,'A');
7          return 0;
8      }
```

.....

.....

Solution

76 65

(h) What will be the output of the given code?

[02 pts]

```
1      #include <stdio.h>
2      int main()
3      {
4          int  A = 10;
5          int B = --A+A++;
6          printf("B is %d\n", B);
7          B=B++;
8          printf("B is %d", B);
9          return 0;
10     }
```

.....

.....

Solution

B is 19

B is 19

(i) What will be the output of the given code?

[02 pts]

```
1      #include <stdio.h>
2      int f1(){
3          return 3;
4      }
5      int f2(){
6          return 4;
7      }
8      int f3(){
9          return 2;
10     }
11     int main(){
12         int s;
13         s=f1()*f2()+f3()*f3();
14         printf("Sum= %d",s);
15         s+=s++*f3()*s--;
16         printf("\nSum= %d",s);
17         return 0;
18     }
```

.....

.....

.....

.....

Solution

Sum=16

Sum=560

2. Fill in the blanks with appropriate words/sentences/syntaxes.

Question 2: 34 pts

Question 2: Marking Scheme

- If the answer does not match with exact code lines or output or formula → **Award ZERO (0)**
- Take care of negative marking of -1 for True/False questions
- for some questions, the theme should be identical to receive marks. eg. Q2(a) & Q2(b).

(a) Consider a two dimensional array **A[3][3]**, and a loop variable **i** is used to denote row and column is denoted by **j**. In this scenario, [01 pts]
arr[i]+j represents. _____.

Solution
address of the element in *ith* row and *jth* column of the array A[3][3]

(b) ***(arr + i)** represents _____. [01 pts]

Solution
base address of *ith* row, or address of the first element *ith* row (j=0, 1st column) of the array A[3][3]

(c) How many substrings can be formed from a character string of length **n**? _____. [01 pts]

Solution
 $n * (n + 1) / 2$

(d) ‘**gets()** is dangerous’ because _____. [01 pts]

Solution
gets() is dangerous because it does not perform bound checking on the size of its input, or causes buffer overflow. (to solve fgets is used).

(e) Difference between void pointer and generic pointer _____. [01 pts]

Solution
Both are same// No difference//identical or other synonyms

(f) Write the C code to declare a two-dimensional dynamic integer array with variable column size of 2 and 4 elements per row. [02 pts]

Solution

```
1      int** A= malloc(sizeof(int) *Row);
2      A[0] = malloc(sizeof(int) * 2);
3      A[1] = malloc(sizeof(int) * 4);
```

(g) Understand the given code snippet and fill the blanks. [02 pts]

```
1      #include <stdio.h>
2      int main()
3      {
4          int (*x)[3];
5          int y[3] = {10,20,30};
6          int *z;
7
8          z = _____; // [1 point]
9          for(int i=0;i<3;i++)
10         printf("%d ",z[i]);
11
12         x = _____; // [1 point]
13         for(int i=0;i<3;i++)
14         printf("%d ",(*x)[i]);
15         return 0;
16     }
```

Solution

```
1      #include <stdio.h>
```

```
2         int main()
3         {
4             int (*x)[3];
5             int y[3] = {10,20,30};
6             int *z;
7
8             z = y; // [1 point]
9             for(int i=0;i<3;i++)
10              printf("%d ",z[i]);
11
12             x = &y; // [1 point]
13             for(int i=0;i<3;i++)
14              printf("%d ",(*x)[i]);
15             return 0;
16         }
```

(h) Complete the following C program that prints itself.

[03 pts]

```
1         int main()
2         {
3             FILE *pt;      char ch;
4             pt = fopen(_____); // [1 point]
5             do
6             {
7                 _____;      [0.5 point]
8             }
9             while (_____); // [1 point]
10            fclose(_____); // [0.5 point]
11            return 0;
12        }
```

.....

.....

.....

.....

Solution

```
1         int main()
2         {
3             FILE *pt;      char ch;
4             pt = fopen(__FILE__, "r"); // [1 point]
5             do
6             {
7                 printf("%c", ch);      [0.5 point]
8             }
9             while ((ch = fgetc(pt)) != EOF); // [1 point]
10            fclose(pt); // [0.5 point]
11            return 0;
12        }
```

- (i) 32-bit IEEE 754 representation of the **octal number 77** is 01000010011111000000000000000000. [03 pts]
- (j) 32-bit IEEE 754 representation of the **decimal number 1000** is 01000100011110100000000000000000. [03 pts]
- (k) 32-bit IEEE 754 representation of the **float value 1.1** is 00111111100011001100110011001101. [03 pts]
- (l) **01000010110010100000000000000000** is the 32-bit IEEE 754 representation of 101 in decimal. [03 pts]
- (m) Our state’s postal abbreviation is ‘BD’. If we interpret 0xBD as a signed 8-bit value (i.e, type char), what is the value in decimal? -67 [03 pts]
- (n) The ASCII value of character ‘Y’ and ‘y’ are 89 and 121 respectively. [02 pts]
- (o) Write **True** or **False** for the following: (1 point for correct, -1 for incorrect)
- i. The associativity of = operator is left to right. False [01 pts]

ii. Using 8-bits, A = 7. The operation (A << 2) results in the value of A to be 27. False [01 pts]

iii. For the given code, the output is Sum= 4. True [01 pts]

```
1         #include <stdio.h>
2         int main(){
3             int s=(2,3,4);
4             printf("Sum= %d",s);
5             return 0;
6         }
```

iv. The below program results in infinite loop. True

[01 pts]

```
1         #include <stdio.h>
2         int main(){
```

```
3           int var=1;
4           var=var--;
5           do {
6               printf("%d", var++);
7           }while (var);
8           return 0;
9       }
```

v. The output of the given code is 33.0000 False [01 pts]

```
1           #include <stdio.h>
2           int main(){
3               float A = 100/3.0;
4               printf("%f",A);
5               return 0;
6           }
```

3. Choose the correct option(s)(2 points for correct, -2 for incorrect).

Question 3: 22 pts

Question 3: Marking Scheme

- Take care of negative marking of -2 for all sub-questions
- Exact answer for grades else deduct.

(a) Which of the following statements about the below code are **incorrect**?: [02 pts]

```
1           struct node
2           {
3               int i;
4               float j;
5               struct node *p;
6           };
7           struct node *s[10];
```

- ☐ **struct node** defines a structure containing an integer, a float, and a pointer to another structure of the same type
- ☒ **The array s can dynamically allocate memory for struct node elements.**
- ☐ The array s can store 10 pointers to **struct node** structures
- ☐ The pointer p in the structure node can point to another structure of type **struct node**.

(b) For the given code, which one of the following expressions, when placed in the blank above, will **NOT** result in a type checking error? [02 pts]

```
1           void f(int, short);
2           void main()
3           {
4               int i = 100;
5               short s = 12;
6               short *p = &s;
7               ----- ; // call to f()
8           }
```

- ☐ f(s,*s)
- ☐ i = f(i,s)
- ☐ f(i,*s)
- ☒ **f(i,*p)**

(c) The most appropriate matching for the following pairs [02 pts]

```
1           W: int *q; printf("value of q, %d",*q);
2           X: m=malloc(5); m= NULL;
3           Y: free(n); n->value = 5;
4           Z: char *p; *p='a';
```

1. using dangling
2. using uninitialized pointers
3. lost memory
4. Segmentation fault
- ☐ X-1 Y-3 Z-2 W4
- ☐ X-2 Y-1 Z-4 W3
- ☐ X-3 Y-4 Z-1 W2
- ☒ **X-3 Y-1 Z-2 W4**

(d) What of the following is the best possible option when the below C program is executed? [02 pts]

```
1           #include <stdio.h>
2           #include <stdlib.h>
3
4           void func() {
5               int *p = malloc(sizeof(int));
6               *p = 42;
```

```

7         printf("Address of p %d", p);
8         free(p);
9         *p = 24;
10        printf("Address of p %d", p);
11    }
12
13    int main() {
14        func();
15        return 0;
16    }

```

- ☐ The program will print the same memory address twice.
☐ The program will crash with a segmentation fault after attempting to modify the freed memory.
☒ **The program will execute without errors, but the behaviour is undefined due to accessing freed memory.**
☐ The program will fail to compile due to the incorrect use of the %d format specifier in the printf function.

(e) The number that will be displayed on execution of the below program is

[02 pts]

```

1    #include <stdio.h>
2    int main(){
3        int arr[]={1,2,3,4,5,6,7,8,9,0,1,2,5}, *ip=arr+5;
4        printf("%d\n", ip[1]);
5        return 0;
6    }

```

- ☐ 6
☒ **7**
☐ 8
☐ 0

(f) What string does the following program print?

[02 pts]

```

1    #include <stdio.h>
2    #include <string.h>
3    void strFunc2 (char A[], int n)
4    {
5        char t;
6        if (n <= 1) return;
7        t = A[0]; A[0] = A[n-1]; A[n-1] = t;
8        strFunc2(&A[1], n-2);
9    }
10   int main ()
11   {
12       char A[10] = "PDS 2005";
13       strFunc2(A, strlen(A));
14       printf("%s", A);
15   }

```

- ☒ **5002 SDP**
☐ 5DS 200P
☐ PDS 2005
☐ SDP 2005

(g) What value does the call h(5) return, where h is defined as follows?

[02 pts]

```

1    int h ( int n )
2    {
3        if (n == 0) return 1;
4        return 2*h(n-1);
5    }

```

- ☐ 25
☒ **32**
☐ 64
☐ 120

(h) What would the below code print?

[02 pts]

```

1    #include <stdio.h>
2    void foo(int n, int sum)
3    {
4        int k = 0, j = 0;
5        if (n == 0) return;
6        k = n % 10;
7        j = n / 10;
8        sum = sum + k;
9        foo (j, sum);
10       printf ("%d,", k);
11   }
12
13   int main ()
14   {

```

```
15         int a = 2048, sum = 0;
16         foo (a, sum);
17         printf ("%d\n", sum);
18
19         getchar();
20     }
```

- ☐ 8, 4, 0, 2, 14
- ☐ 8, 4, 0, 2, 0
- ☐ 2, 0, 4, 8, 14
- ☒ 2, 0, 4, 8, 0

(i) What would the below code print?

[02 pts]

```
1      #include <stdio.h>
2      int f(int n)
3      {
4          static int r = 0;
5          if (n <= 0) return 1;
6          if (n > 3)
7          {
8              r = n;
9              return f(n-2)+2;
10         }
11         return f(n-1)+r;
12     }
13
14     int main()
15     {
16         printf ("%d", f(5));
17     }
```

- ☐ 5
- ☐ 7
- ☒ 18
- ☐ 9

(j) The value returned by f(1) is (for below code)

[02 pts]

```
1      #include <stdio.h>
2      int f(int n)
3      {
4          static int i = 1;
5          if (n >= 5)
6              return n;
7          n = n+i;
8          i++;
9          return f(n);
10     }
11     int main()
12     {
13         printf ("%d", f(1));
14     }
```

- ☐ 5
- ☐ 6
- ☒ 7
- ☐ 8

(k) Consider the following recursive C function. If get(6) function is being called in main() then how many times will the get() function be invoked before returning to the main()? [02 pts]

```
1      void get (int n)
2      {
3          if (n < 1) return;
4          get(n-1);
5          get(n-3);
6          printf ("%d", n);
7      }
```

- ☐ 15
- ☒ 25
- ☐ 35
- ☐ 45

(Don't let fear get the better of you. All the best!)

Space for Rough Work

Space for Rough Work